

Telemetry Stream Auditor

Python Programming Assignment Documentation

Navneet Das

M.Tech Artificial Intelligence

Bennett University

Enrollment: BU26MTAIN100405

August 24, 2026

Assignment Overview

This document outlines the architecture, execution flow, and logical trace for the Telemetry Stream Auditor script. The script is designed to process simulated sensor telemetry batches, monitor data for numerical anomalies, handle simulated data corruption, and execute emergency system stops.

1 Logic Trace

The following trace maps exactly how the execution flows through the nested loops and conditionals for the provided dataset.

Logic Trace

[Batch 0]: [22.5, 23.0, 22.8]

- Iterates through numerical values smoothly.
- No values exceed 35.0, and no differences between adjacent readings exceed 5.0.
- Inner loop completes naturally -> inner 'else' triggers 'continue' -> outer loop proceeds.

[Batch 1]: [25.1, 'ERR', 28.9]

- 25.1 is processed normally.
- 'ERR' triggers the noise handler: prints an ignore message and resets the 'delta' variable to None to prevent false spike detections.
- 28.9 is processed normally.
- Inner loop completes -> inner 'else' triggers 'continue' -> outer loop proceeds.

[Batch 2]: [30.2, 35.5, 40.1]

- 30.2 processed normally.
- 35.5 triggers a SPIKE ($35.5 - 30.2 = 5.3$) & ANOMALY (> 35.0).
- 40.1 triggers a SPIKE ($40.1 - 35.5 = 4.6$) & ANOMALY (> 35.0).
- Inner loop completes -> inner 'else' triggers 'continue' -> outer loop proceeds.

[Batch 3]: [22.0, 22.1, 'STOP']

- 22.0 and 22.1 are processed normally.
- 'STOP' triggers the emergency shutdown condition.
- Inner 'break' executes, terminating the inner loop prematurely.
- Because the inner loop broke, the inner 'else' block is SKIPPED.
- The outer 'break' executes, terminating the entire outer loop.
- Because the outer loop broke prematurely, the outer 'else' block is SKIPPED, meaning "NO system-wide failures" is NOT printed.

2 Architecture & Logic Write-Up

1. State Tracking (`delta`)

The script initializes a `delta` variable for each new batch. This variable stores the previous reading to calculate the difference (spike) between consecutive readings. If an error string occurs, `delta` is reset to `None` so the next valid reading isn't incorrectly compared against an outdated value.

2. Conditional Rule Set

The auditing constraints are handled by specific condition blocks within the inner loop:

- **Type Checking:** It uses `isinstance(reading, (int, float))` to ensure mathematical operations are only performed on valid numerical data.
- **Absolute Anomalies:** Any temperature reading exceeding `35.0` triggers an anomaly warning. (*Note: The script outputs `batch+1` for these alerts, shifting the 0-index to a 1-index for human readability.*)
- **Relative Spikes:** If the difference between the current reading and the immediate previous reading (the `delta`) exceeds `5.0`, a spike alert is printed.
- **Error Handling:** The string "ERR" acts as simulated noise. The script catches it, notifies the user, and skips to the next reading cleanly.

3. Nested Loop Control (The `for...else` Construct)

The script utilizes Python's unique `for...else` loop control structure to manage the "STOP" command across multiple nested loops.

- When "STOP" is encountered, the inner loop executes a `break`.
- By design, a Python `for` loop's `else` block only runs if the loop completes *without* hitting a `break`. Therefore, the inner `break` causes the inner `else: continue` to be skipped.
- Execution falls through to the outer `break`, terminating the entire auditing process instantly.
- Because the outer loop was also terminated via a `break`, the final `else` statement at the bottom of the script is suppressed, correctly acknowledging that a system shutdown occurred rather than completing successfully.